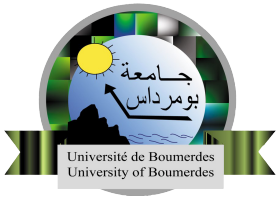
FPGA-Accelerated Network IDS with RISC-V Co-Processor Integration



Youcef Slimani
Salah Eddine Laifa

Institute of Electrical and Electronic Engineering

Fundamental teaching department

M'Hamed Bougara University of Boumerdes

Submitted in partial satisfaction of the requirements for the
Degree of Bachelor
in Electrical and Electronic Engineering

Supervisor Dr.Touzzout Walid

May 2026

Dedications

This thesis is dedicated to our lovely parents for their endless support and encouragement and to our dear brothers. We further extend our dedication to all members of the family.

We dedicate this work also without exception to all our friends, colleagues, and teachers.

Last and not least, I dedicate our work to everyone who has taught, encouraged, and advised us during all our studies.

Acknowledgments

We would like to express my deepest gratitude to our supervisor, Dr.Touzzout Walid, for his invaluable guidance, patience, and constant encouragement throughout this project. Their expertise and insights were instrumental in the completion of this work.

I would also like to thank the IGEE(ex.inelec) Crew for providing the necessary facilities and resources to conduct this project.

Finally, We are forever grateful to our families, especially our parents, for their unwavering love and belief in us during this long journey.

Abstract

This project presents an integrated hardware-software system combining a pipelined RISC-V processor with an FPGA-accelerated network intrusion detection system (IDS). The work is divided into two complementary components: a 32-bit RISC-V processor (BigDCore) implementing the RV32I Base Integer ISA, and a multilayered hardware IDS running on the programmable logic of a Zynq-7020 SoC. The RISC-V processor was developed using VHDL and verified through simulation with C programs of increasing complexity, demonstrating correct execution of the full RV32I instruction set. The IDS implements a layered pipeline architecture processing network packets in real time, detecting 26 attack patterns across Ethernet, IPv4, TCP, and UDP protocols. The system employs a novel co-design approach where the RISC-V processor adapts IDS rules at runtime through a memory-mapped register interface, enabling dynamic threat scoring and adaptive sensitivity tuning. Integration is achieved via AXI-Stream DMA between the Zynq Processing System (which receives Ethernet packets) and the Programmable Logic fabric (which executes the IDS pipeline). Both components are fully verified through simulation, and the combined system demonstrates the feasibility of custom RISC-V implementations integrated with hardware-accelerated security processing for next-generation embedded systems.

Table of Contents

1 Introduction **1**

2 Theoretical Background **2**

2.1 Introduction 2

2.2 RISC-V Instruction Set Architecture 2

2.2.1 History and Origin 2

2.2.2 Design Philosophy 3

2.2.3 RV32I Instruction Formats 3

2.2.4 Processor Design Concepts 4

Von Neumann vs. Harvard Architecture 4

Single-Cycle vs. Pipelined Execution 5

3 System Architecture and Design **7**

3.1 Introduction 7

3.2 Top-Level Architecture 7

3.3 Pipeline Overview 9

3.4 Instruction Fetch (IF) Stage 9

3.5 Instruction Decode (ID) Stage 10

3.6 Execute (EX) Stage 11

3.6.1 ALU Input Selection 11

3.6.2 ALU Operation 12

3.6.3 Branch Address Computation 12

3.6.4 Branch Resolution 12

3.7 Memory Access (MEM) Stage 13

3.8 Write-Back (WB) Stage 13

3.9 Hazard Unit 14

3.9.1 Data Hazards and Forwarding 14

3.9.2 Load-Use Hazards 15

3.9.3 Control Hazards 15

3.10 Conclusion 16

4 Implementation **17**

4.1 Introduction 17
4.2 VHDL Design Organization 17
4.3 Pipeline Register Implementation 18
4.4 Control Unit Implementation 18
4.4.1 Control Bus Pipelining 19
4.5 Datapath Implementation 20
4.5.1 Fetch Stage 20
4.5.2 Decode Stage 20
4.5.3 Execute Stage 21
4.5.4 Memory Stage 21
4.5.5 Write-Back Stage 21
4.6 Hazard Unit Implementation 22
4.6.1 Forwarding Logic 22
4.6.2 Load-Use Stall 22
4.6.3 Control Hazard Flush 23
4.7 Toolchain and Program Loading 23
4.8 Conclusion 25

5 Verification & Testing **26**

5.1 Introduction 26
5.2 Simulation Environment: 26
5.2.1 Testbench Code: 26
5.2.2 Memory Loading 28
5.2.3 Running The Simulation 28
5.3 Tests 28
5.3.1 Test N:°1 28
5.3.2 Test N:°2 29
5.4 Conclusion 30

6 Conclusion **32**

List of Figures

2.1 RV32I Instruction Formats 3

3.1 RV32ICore Top-Level Architecture 8
3.2 RV32ICore IF Stage 10
3.3 RV32ICore ID Stage 11
3.4 RV32ICore EX Stage 13
3.5 RV32ICore MEM Stage 13
3.6 RV32ICore WR Stage 14
3.7 RV32ICore Hazard Unit 15

5.1 R32ICore Initialization 29
5.2 Sum Array Result 29
5.3 Second Test Results 30

Abbreviations

ALU	Complex Instruction Set Computer
BRAM	Block RAM
CISC	Complex Instruction Set Computer
DMA	Direct Memory Access
FCS	Frame Check Sequence
GCC	GNU Compiler Collection
NOP	No Operation
GigE	Gigabit Ethernet
HDL	Hardware Description Language
IDS	Intrusion Detection System
IHL	Internet Header Length
ISA	Instruction Set Architecture
LUT	Look-Up Table
MAC	Media Access Control
MOSFET	Metal-Oxide-Semiconductor Field-Effect Transistor
MREQ	Memory Request
PCB	Printed Circuit Board
PCI	Physical Cell ID
PHY	Physical Interface
PL	Programmable Logic
PS	Processing System
RAM	Random Access Memory
RISC	Reduced Instruction Set Computer
RMII	Reduced Media-Independent Interface

RTL	Register Transfer Level
RV32I	RISC-V 32-bit Base Integer ISA
SoC	System-on-Chip
TCP	Transmission Control Protocol
VHDL	VHSIC Hardware Description Language
XMAS	Christmas Scan

Chapter 1

Introduction

A processor is a specialized digital circuit designed to execute instructions, perform mathematical and logical calculations, and control data flow within a computer system. Over the past decades, microprocessors have not only become a defining trend in the technology industry but have fundamentally reshaped it. As a result, studying and advancing processor design remains a priority for both industry and academia.

RISC-V (Reduced Instruction Set Computer - Five) is an open-standard instruction set architecture (ISA) developed in 2010 at UC Berkeley by Krste Asanović, Andrew Waterman, and Yunsup Lee under the supervision of David Patterson. What makes it unique is its open-standard nature, allowing users to adopt it freely and even modify it, fostering rapid adoption across both academic and commercial domains. Additionally, its adherence to the RISC philosophy makes it not only easier to design and implement but also inherently energy efficient.

This project presents BigDCore (Big Dream Core), a pipelined 32-bit processor implementing the full RV32I Base Integer ISA, described in VHDL and targeting simulation-based verification. RV32I was chosen as the target ISA for its completeness as a base integer instruction set while remaining tractable for a single academic project. Testing is conducted using the industry-standard simulation tool ModelSim, and future work includes the privileged architecture to enable an operating system to run on the processor.

Chapter 2

Theoretical Background

2.1 Introduction

This chapter presents the theoretical foundations underlying the design and implementation of BigDCore. It covers three principal areas: the RISC-V instruction set architecture, fundamental processor design concepts, and an overview of existing RISC-V soft-core implementations. Together, these establish the academic and technical context within which this project is situated.

2.2 RISC-V Instruction Set Architecture

2.2.1 History and Origin

The RISC-V ISA was conceived in 2010 at the University of California, Berkeley, as part of the fifth generation of RISC research projects conducted at the institution. It was developed by Krste Asanović, Andrew Waterman, and Yunsup Lee under the supervision of David Patterson, one of the pioneers of the original RISC architecture. Unlike previous ISA efforts, RISC-V was designed from the outset to be open, stable, and free from proprietary constraints, making it suitable for both academic research and commercial deployment. In 2015, the RISC-V Foundation was established to govern the specification, which was later transferred to RISC-V International in 2020 to ensure its long-term neutrality and global accessibility.

2.2.2 Design Philosophy

RISC-V adheres to the core principles of Reduced Instruction Set Computing. Its design philosophy prioritizes simplicity, regularity, and extensibility. The base ISA is intentionally kept minimal, containing only the instructions necessary for general-purpose computation, while optional standard extensions allow the ISA to be tailored for specific application domains such as floating-point arithmetic, atomic operations, compressed instructions, and vector processing. This modularity ensures that a processor designer can implement only the subset relevant to their use case without carrying the overhead of unused hardware.

A key distinguishing characteristic of RISC-V is its fixed 32-bit instruction width in the base integer ISA, which simplifies instruction fetch and decode logic. All instructions follow a small number of regular formats, and frequently used fields such as the source and destination register indices appear at fixed bit positions across all formats, reducing decoder complexity.

2.2.3 RV32I Instruction Formats

The RV32I base integer ISA defines six instruction formats, each encoding a different combination of operands and immediate values while maintaining a fixed 32-bit instruction width.

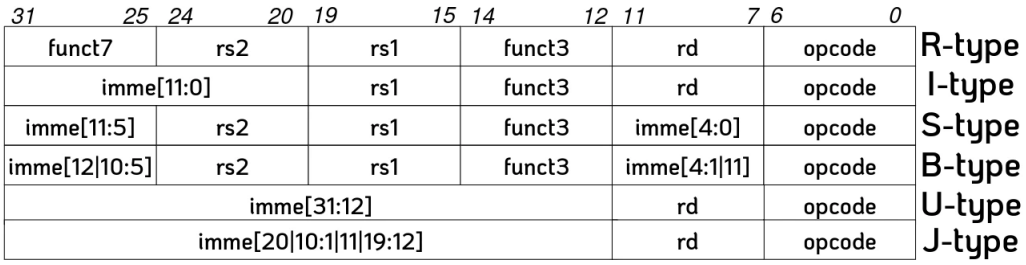


Figure 2.1: RV32I Instruction Formats

The **R-type** format is used for register-to-register arithmetic and logical operations. It encodes two source registers (rs1, rs2) and one destination register (rd), along with

a 7-bit opcode, a 3-bit funct3 field, and a 7-bit funct7 field that together uniquely identify the operation.

The **I-type** format is used for immediate arithmetic operations, load instructions, and environment calls. It encodes one source register (rs1), one destination register (rd), and a 12-bit signed immediate value.

The **S-type** format is used exclusively for store instructions. It encodes two source registers and a 12-bit signed immediate that is split across two non-contiguous fields in the instruction word, a deliberate design choice to keep the register fields at fixed positions.

The **B-type** format is used for conditional branch instructions. It encodes two source registers and a 13-bit signed immediate representing a PC-relative branch offset. Like the S-type, the immediate bits are shuffled to preserve fixed register field positions.

The **U-type** format is used for instructions that operate on upper immediates, namely LUI (Load Upper Immediate) and AUIPC (Add Upper Immediate to PC). It encodes a 20-bit immediate and a destination register.

The **J-type** format is used exclusively by the JAL (Jump and Link) instruction. It encodes a 21-bit PC-relative jump offset and a destination register that receives the return address.

2.2.4 Processor Design Concepts

Von Neumann vs. Harvard Architecture

Two fundamental memory architectures underpin modern processor design. The von Neumann architecture, proposed by John von Neumann in 1945, uses a single shared memory space and a single bus to carry both instructions and data. While this simplifies the memory system and allows flexible allocation of memory between code and data, it introduces a structural bottleneck known as the von Neumann

bottleneck: the processor cannot fetch an instruction and access data simultaneously, limiting throughput.

The Harvard architecture addresses this limitation by providing physically separate memories and buses for instructions and data. This allows simultaneous access to both, eliminating the structural hazard present in von Neumann designs and improving pipeline throughput. The cost is a less flexible memory system, as instruction and data memory cannot share capacity.

A Modified Harvard architecture is a practical compromise widely adopted in modern processors. It maintains separate instruction and data caches at the first level, providing the performance benefits of Harvard architecture, while using a unified main memory accessible by both, preserving the flexibility of von Neumann memory management.

BigDCore adopts a Harvard architecture with fully separate instruction and data memories. This design decision directly supports the five-stage pipeline by ensuring that instruction fetch and data memory access never contend for the same resource.

Single-Cycle vs. Pipelined Execution

A single-cycle processor completes the execution of one instruction per clock cycle by routing it through all functional units in a single pass. While conceptually simple, this approach requires the clock period to be long enough to accommodate the slowest possible instruction, resulting in poor clock frequency and underutilization of hardware during the execution of faster instructions.

A pipelined processor overlaps the execution of multiple instructions by dividing the datapath into discrete stages, each handling one phase of execution. While an individual instruction still takes multiple cycles to complete, the throughput approaches one instruction per cycle under ideal conditions, and the clock period is determined by the slowest pipeline stage rather than the slowest instruction. This trade-off introduces pipeline hazards — structural, data, and control hazards — which must be managed through stalling, forwarding, and flushing mechanisms.

BigDCore implements a classical five-stage pipeline comprising the Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write-Back (WB) stages. A hazard detection unit manages data hazards through stalling and forwarding, and control hazards arising from branch instructions are resolved through pipeline flushing.

Chapter 3

System Architecture and Design

3.1 Introduction

This chapter presents the architecture and design of RV32ICore, a pipelined 32-bit RISC-V processor implementing the RV32I Base Integer ISA. The chapter begins with a top-level overview of the processor's major units, followed by a detailed description of each pipeline stage, the pipeline registers connecting them, and the hazard unit responsible for maintaining correct execution in the presence of data and control hazards.

3.2 Top-Level Architecture

At the highest level of abstraction, RV32ICore is composed of two primary units: the Datapath and the Control Unit, along with a dedicated Hazard Unit that operates across pipeline stages.

The Datapath is responsible for processing and routing data throughout the processor. Its inputs include control signals generated by the Control Unit, instructions fetched from instruction memory, and data read from data memory. Its outputs include the Program Counter (PC) supplied to instruction memory, data written to data memory, and status information such as instruction fields and condition flags fed back to the Control Unit.

The Control Unit is responsible for managing the correct execution of instructions by interpreting instruction fields and generating the appropriate control signals for each pipeline stage. Its inputs are the opcode (bits [6:0]), Funct3 (bits [14:12]), and Funct7 bit 30, along with the processor clock and reset. Its outputs are the set of control signals that govern the behavior of multiplexers, the register file, the ALU, and the data memory throughout the datapath.

The Hazard Unit monitors the pipeline state and intervenes when hazard conditions are detected. It generates stall and flush signals to preserve correct program execution in the presence of data dependencies and control flow changes. Its operation is described in detail in Section 3.9.

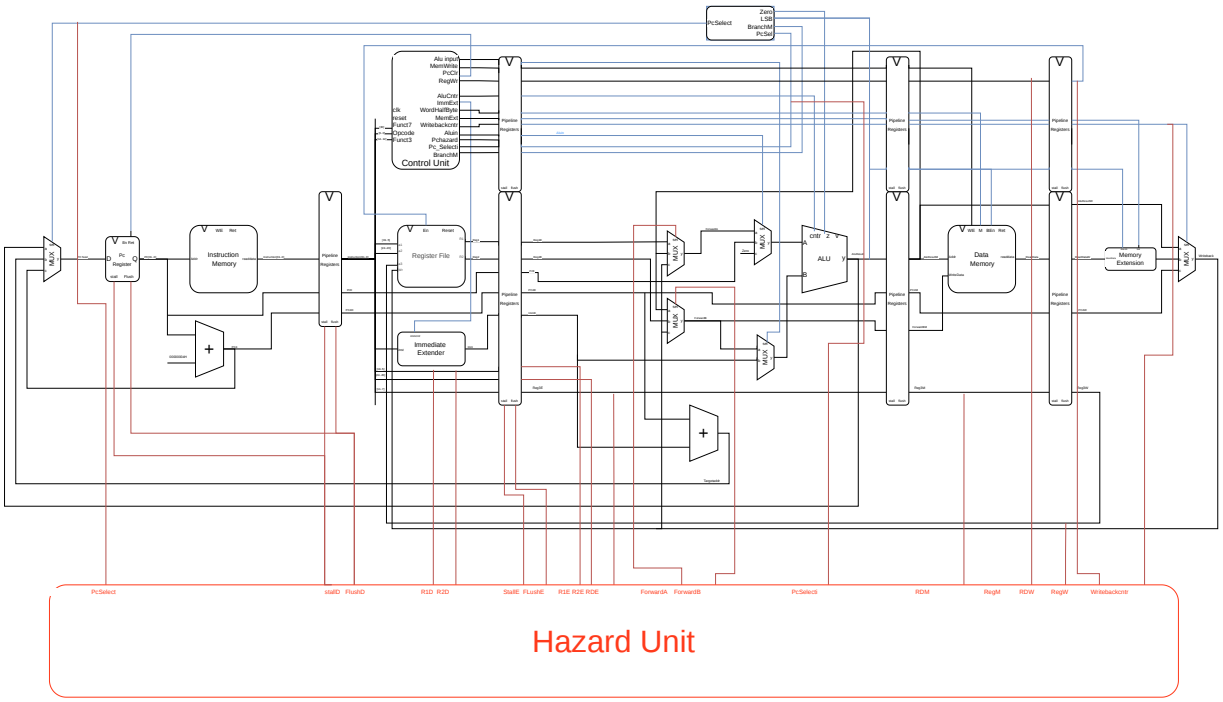


Figure 3.1: RV32ICore Top-Level Architecture

3.3 Pipeline Overview

RV32ICore implements a classical five-stage pipeline. Pipelining allows multiple instructions to be in different stages of execution simultaneously, improving throughput by overlapping work across stages. The five stages are:

- **IF** — Instruction Fetch
- **ID** — Instruction Decode
- **EX** — Execute
- **MEM** — Memory Access
- **WB** — Write-Back

Each stage is separated from the next by a pipeline register that captures the outputs of the current stage and holds them stable for the following stage on the next clock edge. All pipeline registers share the same clock as the rest of the processor and are controlled by two signals: a synchronous reset that clears the register to zero, and a NOP insertion signal that loads a harmless no-operation instruction into the register without clearing its other fields, used during flush operations.

3.4 Instruction Fetch (IF) Stage

The Instruction Fetch stage is responsible for retrieving the current instruction from instruction memory and advancing the Program Counter

The PC is stored in a dedicated register that updates on every rising clock edge under normal operation. Its value is used directly as the address supplied to the instruction memory, which responds combinationally with the 32-bit instruction word stored at that address.

A dedicated adder computes PC+4, representing the address of the next sequential

instruction. This value is carried forward through the pipeline and is used in the Write-Back stage as the return address for jump instructions.

On a taken branch, the PC is updated to the branch target address computed in the Execute stage in the same cycle that the flush signal is issued. Under normal sequential execution, the PC is updated to PC+4.

The instruction memory is separate from the data memory, consistent with the Harvard architecture adopted by RV32ICore. This ensures that instruction fetch never contends with data memory access, eliminating structural hazards between the IF and MEM stages.

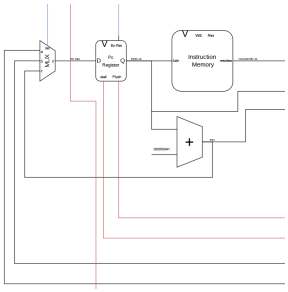


Figure 3.2: RV32ICore IF Stage

3.5 Instruction Decode (ID) Stage

The Instruction Decode stage deconstructs the fetched instruction and prepares the operands and control signals required for execution.

The 32-bit instruction word is partitioned into its constituent fields according to the RV32I specification. The source register addresses rs1 and rs2 are located at bits [19:15] and [24:20] respectively, and the destination register address rd is located at bits [11:7]. These addresses are wired directly to the read ports of the register file, which responds combinationally with the contents of the addressed registers. Register x0 is hardwired to zero; any read of x0 returns zero and any write to x0 is silently discarded.

An Immediate Extension unit receives the full 32-bit instruction word and produces

a 32-bit sign-extended immediate value. The extension logic selects the appropriate immediate format — I, S, B, U, or J — based on the opcode, and assembles the immediate from the relevant instruction bits according to the RV32I encoding specification.

The opcode (bits [6:0]), Funct3 (bits [14:12]), and Funct7 bit 30 are forwarded to the Control Unit, which decodes them and generates the full set of control signals for the instruction. These control signals are stored in the ID/EX pipeline register alongside the register data and immediate value and propagate through the pipeline with the instruction.

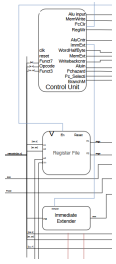


Figure 3.3: RV32ICore ID Stage

3.6 Execute (EX) Stage

The Execute stage performs the arithmetic, logical, or address computation required by the current instruction. It is the most complex stage of the datapath, containing the ALU, two input multiplexers, a branch adder, and the forwarding multiplexer logic.

3.6.1 ALU Input Selection

Two multiplexers select the operands presented to the ALU inputs.

MuxX3 selects ALU input A from three possible sources: the value of register rs1, the current PC (used by AUIPC), or zero (used by LUI, which adds the upper immediate to zero so that the ALU result path is reused without requiring dedicated

hardware). The selection is governed by a control signal generated by the Control Unit.

MuxX2 selects ALU input B from two possible sources: the value of register rs2 (for register-to-register instructions) or the sign-extended immediate (for immediate-type instructions). The selection is governed by a corresponding control signal.

3.6.2 ALU Operation

The ALU receives the two selected operands and performs the operation specified by the ALU control signal, which is derived from the opcode, Funct3, and Funct7 fields decoded in the Control Unit. Supported operations include addition, subtraction, bitwise AND, OR, and XOR, logical and arithmetic shift, and signed and unsigned comparison. The ALU produces a 32-bit result and a zero flag used for branch condition evaluation.

3.6.3 Branch Address Computation

A dedicated adder computes the branch target address by summing the PC value carried from the IF stage with the sign-extended immediate. This target address is passed to the PC register when a branch is taken.

3.6.4 Branch Resolution

Branch conditions are evaluated in the Execute stage. When a branch is taken, the Hazard Unit issues a flush signal that inserts NOP instructions into the IF/ID and ID/EX pipeline registers in the same cycle that the PC is updated to the branch target. This discards the two incorrectly fetched instructions that entered the pipeline after the branch. No branch prediction is employed; the pipeline always flushes on a taken branch.

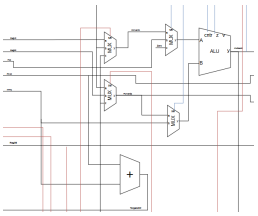


Figure 3.4: RV32ICore EX Stage

3.7 Memory Access (MEM) Stage

The Memory Access stage handles Load and Store instructions by interfacing with the data memory.

For Store instructions, the address computed by the ALU in the Execute stage is used as the write address, and the value of register rs2 — carried through the pipeline — is written to data memory. For Load instructions, the same computed address is used as the read address, and the data memory responds with the value stored at that location.

The data memory supports three access widths controlled by a signal derived from the Funct3 field: byte (8-bit), halfword (16-bit), and word (32-bit). For instructions that do not access memory, this stage passes the ALU result and control signals through to the Write-Back stage without interacting with the data memory.

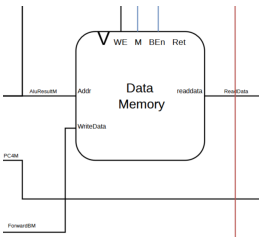


Figure 3.5: RV32ICore MEM Stage

3.8 Write-Back (WB) Stage

The Write-Back stage selects the value to be written to the destination register and commits it to the register file.

A Memory Extension unit first processes the raw data returned by the data memory. It performs sign extension or zero extension of byte and halfword values according to the access type specified by the Funct3 field, ensuring that the value written to the 32-bit register file conforms to the RV32I specification for load instructions.

A multiplexer then selects among three possible write-back sources: the ALU result (for arithmetic, logical, and address computation instructions), the extended memory data (for load instructions), and PC+4 (for JAL and JALR, which write the return address to the destination register). The selected value is written to the register file at the address specified by rd on the rising clock edge, completing the instruction's execution.

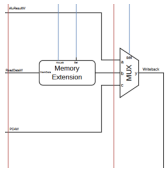


Figure 3.6: RV32ICore WR Stage

3.9 Hazard Unit

The Hazard Unit monitors the pipeline registers and intervenes when conditions arise that would cause incorrect execution. It handles three categories of hazards: data hazards, load-use hazards, and control hazards.

3.9.1 Data Hazards and Forwarding

Data hazards occur when an instruction depends on the result of a preceding instruction that has not yet been written back to the register file. RV32ICore resolves most data hazards through forwarding, which routes computed results directly from later pipeline stages back to the ALU inputs in the Execute stage, bypassing the register file.

The Hazard Unit compares the destination register of instructions in the EX/MEM

and MEM/WB pipeline registers against the source registers of the instruction currently in the Execute stage. When a match is detected, forwarded values are selected at both MuxX2 and MuxX3 in place of the register file outputs. When forwarding from the MEM/WB stage following a load instruction, the forwarded value is taken from the output of the Memory Extension unit, ensuring that the correctly sign-extended loaded value is used.

3.9.2 Load-Use Hazards

A load-use hazard occurs when a load instruction is immediately followed by an instruction that uses the loaded value. Since the loaded data is not available until the end of the Memory Access stage, forwarding alone cannot resolve this hazard. The Hazard Unit detects this condition by checking whether the instruction in the Execute stage is a load and whether its destination register matches a source register of the instruction in the Decode stage. When detected, the Hazard Unit stalls the pipeline for one cycle by freezing the IF/ID and ID/EX pipeline registers and inserting a NOP into the Execute stage, allowing the loaded value to reach the MEM/WB stage where it can be forwarded.

3.9.3 Control Hazards

Control hazards arise from branch and jump instructions, whose target addresses are not known until the Execute stage. Since two instructions are fetched after every branch before its outcome is resolved, those instructions must be discarded if the branch is taken. The Hazard Unit handles this by issuing a flush signal that inserts NOP instructions into the IF/ID and ID/EX pipeline registers in the same cycle that the PC is updated to the branch target address.

Figure 3.7: RV32ICore Hazard Unit

3.10 Conclusion

This chapter has presented the complete architecture of RV32ICore, covering its top-level decomposition into Datapath, Control Unit, and Hazard Unit, and providing a detailed description of each pipeline stage and the registers connecting them. The five-stage Harvard pipeline provides a structured and efficient framework for RV32I instruction execution. The forwarding and stalling mechanisms implemented in the Hazard Unit ensure correct execution in the presence of data dependencies, while pipeline flushing resolves control hazards introduced by branch and jump instructions. The architectural decisions described in this chapter form the direct basis for the VHDL implementation presented in the following chapter.

Chapter 4

Implementation

4.1 Introduction

This chapter describes the implementation of RV32ICore in VHDL, covering the structural organization of the design, the implementation of each major component, key design decisions made during development, and the toolchain used to compile and load test programs into the simulation environment. The implementation follows directly from the architecture described in Chapter 3, with each architectural unit realized as a distinct VHDL entity.

4.2 VHDL Design Organization

RV32ICore is organized as a hierarchy of VHDL entities, each encapsulating a well-defined functional unit. The top-level entity *P_Processor_Hazard* instantiates three major components: the datapath *(P_Datapath)*, the control unit *(P_Control_Unit)*, and the hazard unit *(Hazard_Unit)*. Memory is intentionally kept external to the processor entity, treated as a peripheral connected through the top-level ports. This separation reflects a deliberate design decision: by exposing the instruction and data memory interfaces at the top level, the processor can be connected to different memory configurations without modifying its internal logic, improving modularity and testability.

The top-level interface exposes the following ports: a clock and reset, a 32-bit in-

struction input, a 32-bit read data input from data memory, and outputs for the program counter, data address, write data, memory write enable, byte enable, and access mode signals. This interface is sufficient to connect RV32ICore to any external Harvard memory system.

4.3 Pipeline Register Implementation

A central implementation decision in RV32ICore is the use of a single generic VHDL component, *RegEn*, to implement all pipeline registers throughout the design. This component is parameterized by data width through a generic and supports four operating modes controlled by its input signals: normal clocked operation, stall (output held, input ignored), flush (output loaded with a predefined NOP value), and synchronous reset (output cleared to zero).

The NOP value inserted during a flush is x"*00000033*", corresponding to the RV32I instruction *ADD x0, x0, x0*. This is a valid instruction that writes to register x0, which is hardwired to zero, making it functionally harmless. Using a valid instruction rather than an all-zero pattern avoids potential undefined behavior in the decode and control logic during pipeline recovery.

Separate width variants of the same component (*RegEn_5bit, RegEn_9bit, RegEn_6bit*) are used for narrower signals such as register addresses and reduced control buses, maintaining the same structural pattern throughout the design.

4.4 Control Unit Implementation

The control unit is implemented as *P_Control_Unit* and is itself structured as a hierarchy of sub-components. It contains three functional blocks: the main control unit (*P_Main_unit*), the ALU control unit (*Alu_cntr*), and the execution flow controller (*Excution_Flow*).

P_Main_unit is a combinational decoder that maps the opcode and Funct3 fields

to the full set of control signals required by the datapath. Its outputs include the immediate extension select, memory access width, register write enable, memory write enable, memory extension control, write-back select, PC select, ALU input select, branch mode, and ALU input override for AUIPC and LUI instructions.

Alu_cntr decodes the opcode, Funct3, and bit 5 of Funct7 to produce the 4-bit ALU operation code. Using only bit 5 of Funct7 is sufficient to distinguish between arithmetic pairs such as ADD/SUB and SRL/SRA, avoiding the need to decode all seven Funct7 bits.

Excution_Flow resolves the final PC select signal in the Execute stage. It receives the initial PC select from the control bus pipeline register, the branch mode encoding, the ALU zero flag, and the ALU LSB, and produces the final 2-bit PC select that determines whether the next PC is PC+4, the ALU result (for JALR), or the branch target address.

4.4.1 Control Bus Pipelining

A notable implementation strategy in the control unit is the aggregation of all control signals into a single control bus that is pipelined through*RegEn* registers in parallel with the datapath pipeline registers. Rather than routing individual control signals through separate pipeline registers, the control signals are packed into a 21-bit bus at the Decode stage output and propagated as a unit through successive pipeline stages. At each stage boundary, only the subset of control signals relevant to that stage is extracted from the bus.

The bus is progressively narrowed as signals are consumed: the full 21-bit bus is reduced to a 9-bit bus entering the Memory stage (retaining only memory and write-back controls), and further reduced to a 6-bit bus entering the Write-Back stage (retaining only register write enable, memory extension control, and write-back select). This approach simplifies the pipeline register instantiation and makes the control signal flow explicit and traceable in the VHDL source.

4.5 Datapath Implementation

The datapath is implemented as *P_Datapath* and instantiates all pipeline stages, pipeline registers, forwarding multiplexers, and functional units. The following describes the implementation of each stage.

4.5.1 Fetch Stage

The PC is stored in a *RegEn* instance configured with the stall input connected to *StallF*. Under normal operation it loads *Pc_Next* on every rising clock edge. The *Pc_Next* signal is selected by a three-input multiplexer (*Mux3*) among three sources: the ALU result (for JALR), PC+4 (for sequential execution), and the branch target address computed in the Execute stage (for taken branches). PC+4 is computed by a dedicated adder that adds the constant four to the current PC. Both the current PC and PC+4 are registered into the IF/ID pipeline register for use in later stages.

4.5.2 Decode Stage

The register file is read combinationally using rs1 and rs2 addresses extracted directly from the pipelined instruction word at bits [19:15] and [24:20] respectively. The destination register address rd is extracted from bits [11:7] and propagated through the pipeline in a dedicated 5-bit pipeline register chain (*Reg_A3_PLR3* through *Reg_A3_PLR5*) alongside the instruction, reaching the Write-Back stage where it addresses the register file write port.

The immediate extension unit receives the full 32-bit instruction word and a 3-bit select signal from the control unit, and produces the appropriate 32-bit sign-extended immediate for the instruction's format.

The register source addresses rs1 and rs2 are also registered separately into Rs1E and Rs2E for use by the Hazard Unit in forwarding comparisons.

4.5.3 Execute Stage

Two forwarding multiplexers (*ForwardingA* and *ForwardingB*) sit at the inputs of the ALU input selection multiplexers. Each is a Mux3 instance that selects among the register file output (no forwarding), the current Write-Back result (forwarding from WB), and the EX/MEM ALU result (forwarding from MEM). The selection is controlled by the 2-bit *ForwardA* and *ForwardB* signals from the Hazard Unit.

The forwarded rs2 value (*ForwaredB*) is also registered into the MEM stage in a dedicated pipeline register (*ForwaredB_PLR4*). This is necessary for Store instructions, which require the value of rs2 at the Memory stage as the data to be written to memory, while the ALU computes the store address using rs1 and the immediate.

ALU input A is selected by *ALU_in_A* (*Mux3*) among the forwarded rs1 value, the PC (for AUIPC), and zero (for LUI). ALU input B is selected by *ALU_in_B* (*Mux2*) between the sign-extended immediate and the forwarded rs2 value.

The branch target address is computed by a dedicated adder summing the PC value carried from the Decode stage with the sign-extended immediate.

4.5.4 Memory Stage

The data address presented to external memory is the ALU result from the EX/MEM pipeline register. The write data is the forwarded rs2 value from *ForwaredB_PLR4*. The two least significant bits of the ALU result (*Alu_LSB_Mem*) are passed to the top level as the ByteEn signal, providing the byte offset within a word to the external memory for sub-word accesses.

4.5.5 Write-Back Stage

The Memory Extension unit (*Mem_SignExt*) receives the raw 32-bit read data from memory, the two LSBs of the ALU result as a byte offset, and the memory extension control signal derived from Funct3. It produces a correctly sign- or zero-extended 32-bit value for byte and halfword load instructions.

The Write-Back multiplexer (*Mux3*) selects among PC+4 (for JAL/JALR return address), the extended memory data (for load instructions), and the ALU result (for all other instructions). The selected value is written to the register file at the address held in *Reg_A3_PLR5*.

4.6 Hazard Unit Implementation

The Hazard Unit is implemented as a fully combinational entity Hazard_Unit. It receives the source register addresses from the Decode and Execute stages, the destination register addresses from the Execute, Memory, and Write-Back stages, the register write enables from the Memory and Write-Back stages, and the write-back control LSB and PC select MSB from the control pipeline.

4.6.1 Forwarding Logic

Forwarding priority is implemented as a priority-encoded conditional check. For each ALU input, the MEM stage forwarding path takes priority over the WB stage path, ensuring that the most recently computed value is always selected in the case of back-to-back dependent instructions. Forwarding is suppressed when the destination register is x0, since x0 is hardwired to zero and forwarding from it would produce incorrect results.

The forwarding condition also checks a combined signal: forwarding is enabled when either *RegWriteM* is asserted or when the LSB of *Pc_Select_Initial* is *'0'*, which identifies JAL and JALR instructions that write PC+4 to a destination register but are not flagged by the standard *RegWrite* path at that pipeline stage.

4.6.2 Load-Use Stall

The load-use stall is detected by checking whether the *Writeback_cntr* LSB is asserted — indicating a load instruction in the Execute stage — and whether the load destination register matches either source register of the instruction currently in the

Decode stage. When detected, *StallF* and *StallD* are asserted to freeze the Fetch and Decode stage pipeline registers, and *FlushE* is asserted to insert a NOP into the Execute stage for one cycle.

4.6.3 Control Hazard Flush

The flush signal for taken branches is derived from the MSB of the *Pc_Select* signal, which is asserted whenever the processor takes a branch or jump. *FlushE* is asserted as the logical OR of the load-use stall and the PC select signal, flushing the Execute stage in both cases. *FlushD* is asserted on the PC select signal alone, flushing the Decode stage on taken branches. This correctly discards the two instructions fetched after the branch while leaving the Fetch stage free to begin fetching from the branch target on the following cycle.

4.7 Toolchain and Program Loading

To verify RV32ICore through simulation, a software toolchain is used to compile C test programs into binary executables targeting the RV32I ISA, which are then converted into a format that can be initialized into the instruction memory for simulation. The following code will be used as an Example:

```
1    // test.c
2
3  // Tests loops and addition
4  int sum_array(int arr[], int len) {
5      int sum = 0;
6      for (int i = 0; i < len; i++) {
7          sum += arr[i];
8      }
9      return sum;
10 }
11 int main() {
12     int arr[] = {3, 7, 2, 9, 1};
```

```
13
14      int s = sum_array(arr, 5);        // expected: 22
15
16      return 0;
17  }
```

The compiler used is the xPack RISC-V Embedded GCC toolchain (*riscv-none-elf-gcc*), configured to target the RV32I base ISA without compressed instructions or floating-point extensions, matching exactly the instruction set implemented by RV32ICore. A custom linker script defines the memory layout, setting the origin of the text segment to address *0x00000000* to match the reset vector of the processor, and configuring the initial stack pointer to the top of the available data memory region.

A startup assembly file initializes the processor state before transferring control to the C main function. It sets the stack pointer register and performs any necessary early initialization before the C runtime environment is available.

```
1      00000000 <_start>:
2     0:   40000113                li      sp,1024
3     4:   11400513                li      a0,276
4     8:   11400593                li      a1,276
5
6   0000000c <bss_loop>:
7     c:   00b55863                bge     a0,a1,1c <bss_done>
8    10:   00052023                sw      zero,0(a0)
9    14:   00450513                addi    a0,a0,4
10   18:   ff5ff06f                j       c <bss_loop>
11
12  0000001c <bss_done>:
13   1c:   07c000ef                jal     98 <main>
14
15  00000020 <halt>:
16   20:   0000006f                j       20 <halt>
```

Note: The compiled code is very large hence a sample is given, and the remaining is illustrated in the Appendix.

4.8 Conclusion

This chapter has described the complete VHDL implementation of RV32ICore, from the top-level entity structure through the detailed implementation of each pipeline stage, the control bus pipelining strategy, the hazard unit combinational logic, and the software toolchain used to generate and load test programs. Several implementation decisions — including the generic RegEn pipeline register component, the progressive narrowing of the control bus, the choice of ADD x0, x0, x0 as the NOP encoding, and the dedicated forwarded rs2 pipeline register for store instructions — reflect deliberate engineering choices made to balance correctness, clarity, and design reuse. The implementation described here is verified through the simulation-based testing methodology presented in the following chapter.

Chapter 5

Verification & Testing

5.1 Introduction

A processor is considered functionally correct when it successfully executes the complete set of target ISA instructions, producing results consistent with the architectural specification. To verify RV32ICore, a set of C programs of increasing complexity are compiled using the xPack RISC-V GCC toolchain and executed in simulation. Correct execution is determined by inspecting the values held in memory or the register file at the end of simulation and comparing them against analytically computed expected results.

5.2 Simulation Environment:

Most of the tests performed on RV32ICore will follow a structured simulation environment. The environment contains:

5.2.1 Testbench Code:

```
1         Library ieee;
2         use ieee.std_logic_1164.all;
3         entity Hazard_Testbench is
4         end entity;
5
6         architecture arch of Hazard_Testbench is
7         component P_Processor_Hazard is
8         port(...
```

```
9         );
10        end component;
11        component RAM_Inst is
12        port(...
13        );
14        end component;
15        component RAM is
16        port(...
17        );
18        end component;
19        Signal clk ,reset: std_logic;
20        Signal instruction, ReadData: std_logic_vector(31 downto 0);
21        Signal MemWrite: std_logic;
22        Signal ByteEn, Mode: std_logic_vector(1 downto 0);
23        Signal Pc: std_logic_vector(31 downto 0);
24        Signal DataAdr, WriteData: std_logic_vector(31 downto 0);
25        Signal clk_cnt, Inst_Change_Cnt: integer := 0;--number Of clocks counter
26        begin
27        UUT: P_Processor_Hazard port map(...
28        );
29        Data_RAM: RAM port map(...
30        );
31        Inst_RAM: RAM_Inst port map(...
32        );
33        clock: process Begin
34        loop
35        clk<= '1';
36        wait for 10 ns;
37        clk<= '0';
38        wait for 10 ns;
39        end loop;
40        end process;
41
42        Clock_counter: process(clk)
43        variable last: std_logic_vector(31 downto 0);
44        Begin
45        if(rising_edge(clk)) then
46        clk_cnt<= clk_cnt + 1;
47        if (instruction /= last) then
48        Inst_Change_Cnt <= Inst_Change_Cnt + 1;
49        end if;
50        last := instruction;
51        end if;
52        end process;
53
54        Stimulus: process Begin
55        reset<= '1';
56        wait for 40 ns;
57        reset<= '0';
58        wait;
59        end process;
60        end architecture;
```

The code maps between the processor and its memories and defines the necessary

signals to do so. The testbench initializes by applying a reset for two clock cycles ensuring that the Pc, register file, and memory are all starting with Zero. A clock of 20 ns period is simulated using a Clock process and wait commands, and a counter is added to calculate the number of instructions preformed in a program used to calculate performance.

5.2.2 Memory Loading

The programs used for testing can be very long making impractical to load them manually. A more efficient way is to use the compiler. The output file can be turned into an output hex file which can be directly loaded into memory using a command supported by ModelSim (mem load -infile output.hex -format hex sim:/hazard_testbench/Inst_RAM/Mem).

5.2.3 Running The Simulation

By running the simulation for an appropriate amount of time the results generated by the processor can be verified in memory where they are stored. Additionally if a it is necessary to check the behavior of any signal during the processing it is possible to do so using Waveforms.

5.3 Tests

Two tests will be preformed one of a summing function called by main and the other is more complex containing multiple arithmetic functions.

5.3.1 Test N:°1

The Processor initialize by resetting for two clock periods the PC starts with address 0 and increment by 4 in Normal sequence (No execution flow instructions) instructions are being fetched as the waveform suggests.

Figure 5.1: R32ICore Initialization

When the execution of the program ends we can check the final values stored in stack due to instruction

```
E4:        sw  a0,-20(s0) ; [000003BC] = a0 where a0= 16H
```

The following Figure indicates that indeed the program executed as expected by computing the value 16H and storing it in location address 0x000003BC, which when divided by the word size of 4 bytes corresponds to word index 0xEF in the data memory. With this result it is appropriate to say that the processor is working

Figure 5.2: Sum Array Result

properly another test will be done for further verification.

5.3.2 Test N:°2

The second test uses the following C code. It contains the previously used array summer it also contains a multiplayer, a pointer manipulator, and a bit-wise manipulator. The analyzed values should be 16H, 78H, 09H, and 08H respectively.

```c
// test.c

// Tests loops and addition
int sum_array(int arr[], int len) {
        int sum = 0;
        for (int i = 0; i < len; i++) {
                sum += arr[i];
        }
        return sum;
}

// Tests recursion and stack depth
int factorial(int n) {
        int result = 1; // multiply by repeated addition
        for (int i = 2; i <= n; i++) {
                int temp = 0;
                for (int j = 0; j < i; j++) {
                        temp += result;
                }
```

```
20                      result = temp;
21              }
22              return result;
23      }
24      // Tests pointer manipulation
25      int find_max(int arr[], int len) {
26              int max = arr[0];
27              for (int i = 1; i < len; i++) {
28                      if (arr[i] > max)
29                      max = arr[i];
30              }
31              return max;
32      }
33
34      // Tests bitwise operations
35      unsigned int count_bits(unsigned int n) {
36              unsigned int count = 0;
37              while (n) {
38                      count += n & 1;
39                      n >>= 1;
40              }
41              return count;
42      }
43
44      int main() {
45              int arr[] = {3, 7, 2, 9, 1};
46              int s = sum_array(arr, 5);       // expected: 22
47              int f = factorial(5);             // expected: 120
48              int m = find_max(arr, 5);        // expected: 9
49              unsigned int b = count_bits(255); // expected: 8
50              return 0;
51      }
```

By checking the contents of the memory we find.

Figure 5.3: Second Test Results

The processor is preforming as intended and further testing was done on the unused instructions by the previous programs.

5.4 Conclusion

This chapter has presented the verification methodology and simulation results for RV32ICore. A structured testing approach was adopted in which C programs of

increasing complexity were compiled using the xPack RISC-V GCC toolchain and executed in the ModelSim simulation environment.

Correct execution was determined by comparing values stored in the data memory against analytically derived expected results, with complete assembly listings provided in the Appendix for reference. Reset behavior was verified to be correct, with the PC initializing to zero and the pipeline filling in proper sequence following reset release. Two principal test programs were used: the first verified loop execution, load and store operations, and basic arithmetic through a sum-of-array computation producing the correct result of 22 (0x16); the second subjected the processor to a more demanding workload involving multiple concurrent function calls, factorial computation, pointer manipulation, and bitwise operations, yielding the correct results of 22, 120, 9, and 8 respectively. The remaining RV32I instructions were individually verified through targeted test programs. The combined results confirm that RV32ICore correctly executes the full RV32I Base Integer ISA under simulation.

Chapter 6

Conclusion

This project has presented the design, implementation, and verification of RV32ICore, a pipelined 32-bit processor implementing the RV32I Base Integer ISA, described in VHDL and verified through simulation.

The work began with a study of the RISC-V ISA and foundational processor design concepts, establishing the academic context within which the project is situated. The architectural design of RV32ICore was then developed, comprising a classical five-stage Harvard pipeline with a dedicated hazard unit responsible for resolving data hazards through forwarding and stalling, and control hazards through pipeline flushing. The implementation translated this architecture directly into a hierarchy of VHDL entities, with notable design decisions including a generic reusable pipeline register component, a progressively narrowed control bus, and an external memory interface that improves modularity. The processor was verified by compiling and executing C programs of increasing complexity in ModelSim, with results confirmed against expected values. The full RV32I Base Integer ISA was demonstrated to execute correctly under simulation.

The project objectives stated in Chapter 1 have been met: a complete RV32I processor has been implemented in VHDL, verified through simulation using compiler-generated test programs, and documented to a standard suitable for academic presentation.

Several directions for future work are identified. The most immediate extension is deployment on an FPGA target, which would allow measurement of actual clock frequency, resource utilization, and real-world performance. Adding the RV32M extension would introduce hardware multiply and divide instructions, significantly broadening the range of programs the processor can efficiently execute. Implementation of the RISC-V privileged architecture specification — specifically Supervisor mode and the virtual memory system — would enable a full operating system to run on the processor, representing a substantial step toward a complete soft-core system. Finally, pipeline optimization through branch prediction or deeper pipelining could improve instruction throughput beyond what the current flush-on-branch approach achieves.